

JAVASCRIPT

1. Introduction to JavaScript

- **What is JavaScript?**
 - History and evolution
 - Client-side vs. server-side scripting
- **Setting up your environment**
 - Browsers and Developer Tools
 - Installing a code editor (e.g., VS Code)
 - Basic debugging in Developer Tools

2. JavaScript Basics

- **Syntax and Fundamentals**
 - Variables (`var`, `let`, `const`)
 - Data types (string, number, boolean, null, undefined, symbol, bigint)
 - Operators (arithmetic, assignment, comparison, logical)
- **Control Structures**
 - Conditionals (`if`, `else`, `else if`, `switch`)
 - Loops (`for`, `while`, `do...while`, `for...of`, `for...in`)
- **Functions**
 - Function declarations and expressions
 - Arrow functions
 - Parameters and return values
 - Scope and closures

3. DOM Manipulation

- **Document Object Model (DOM)**
 - Selecting elements (`getElementById`, `querySelector`, etc.)
 - Modifying HTML and CSS (`innerHTML`, `style`, `classList`)
 - Event handling (`addEventListener`, event types)
- **Forms and Input**
 - Form validation
 - Event listeners for forms
 - Dynamic form inputs

4. Intermediate Concepts

- **Arrays and Objects**
 - Creating and manipulating arrays
 - Array methods (`map`, `filter`, `reduce`, `forEach`)
 - Working with objects and object methods
- **Error Handling**
 - `try...catch`
 - `throw` statements
 - Custom error messages
- **Asynchronous JavaScript**
 - Callbacks
 - Promises (`then`, `catch`, `finally`)
 - `async/await`

5. Advanced Topics

- **ES6+ Features**
 - Destructuring
 - Template literals
 - Default parameters
 - Rest and spread operators
 - Modules (`import` and `export`)
- **Object-Oriented Programming (OOP)**
 - Prototypes and inheritance
 - Classes and constructors
 - Encapsulation, inheritance, and polymorphism
- **Functional Programming**
 - Pure functions
 - Higher-order functions
 - Immutability
 - Currying and partial application

6. Working with APIs

- **AJAX and Fetch API**
 - Making HTTP requests
 - Consuming APIs
 - Error handling in API calls
- **JSON**
 - Parsing and stringifying JSON data
 - Working with API responses

7. Browser-Specific Features

- **LocalStorage, SessionStorage, and Cookies**
- **Geolocation API**
- **Web Workers**
- **Canvas and SVG**

8. Testing and Debugging

- **Debugging Techniques**
 - Using breakpoints
 - The `console` object (`log`, `warn`, `error`, `table`)
- **Testing Frameworks**
 - Unit testing with Jest or Mocha
 - Writing test cases

9. JavaScript Build Tools

- **Introduction to npm and Yarn**
- **Module bundlers**
 - Webpack
 - Parcel
- **Transpilers**
 - Babel

10. Frameworks and Libraries

- **Introduction to Popular Frameworks**
 - React.js
 - Vue.js
 - Angular
- **Utility Libraries**
 - Lodash
 - Moment.js (or alternatives like Day.js)

11. Real-World Applications

- **Building Projects**
 - To-Do List
 - Weather App (using an API)
 - Portfolio Website
 - E-commerce Cart
 - Real-time Chat App
- **Version Control**
 - Git basics

- Hosting projects on GitHub

12. Performance Optimization

- **Optimizing JavaScript**
 - Minification and bundling
 - Lazy loading
 - Debouncing and throttling
- **Web Performance APIs**
 - Timing APIs
 - Analyzing performance in DevTools

13. Advanced Topics (Optional)

- **TypeScript**
 - Introduction and usage with JavaScript
- **Node.js**
 - Setting up a backend
 - Creating REST APIs
- **WebAssembly**
 - Introduction and use cases

REACT NATIVE

(Mobile App Development combo-Javascript+React Native)

1. Prerequisites

Before diving into React Native, ensure you have:

1. **JavaScript Proficiency**
 - ES6+ features: `let/const`, arrow functions, destructuring, template literals, promises, and `async/await`.
 - Understanding of arrays, objects, functions, and event handling.
2. **React Fundamentals**
 - JSX syntax, components (functional and class-based), props, state, and lifecycle methods/hooks.
3. **Basic Mobile App Concepts**
 - Familiarity with native mobile development concepts like navigation, touch gestures, and platform differences (iOS vs Android).

2. Introduction to React Native

- **What is React Native?**
 - Overview of React Native and its advantages.
 - How React Native differs from React and native mobile app development.
- **Setting Up the Environment**
 - Installing Node.js, npm/yarn, and Expo CLI.
 - Setting up Android Studio and Xcode for native builds.
 - Exploring the folder structure of a React Native project.

3. Core Concepts

- **React Native Components**
 - Core components: `View`, `Text`, `Image`, `ScrollView`, `TextInput`, `FlatList`, `SectionList`.
 - Using `StyleSheet` for styling.
- **Flexbox for Layout**
 - Understanding Flexbox properties in React Native.
 - Building responsive layouts.
- **Handling User Input**
 - Working with `Button`, `TouchableOpacity`, and gestures.
 - Handling user events like taps, swipes, and scrolls.

4. Navigation in React Native

- **React Navigation**
 - Installing and setting up React Navigation.
 - Stack, Tab, Drawer, and Nested navigators.
 - Passing parameters between screens.
 - Customizing navigation headers.
- **Deep Linking**
 - Understanding deep linking in React Native.
 - Configuring deep links for Android and iOS.

5. State Management

- **Using State and Props**
 - Managing local component state with `useState`.
 - Passing data between components via props.
- **Context API**
 - Using React Context for global state management.
- **Advanced State Management**
 - Integrating Redux or Zustand for larger apps.
 - Using Redux Toolkit for simplifying state management.

6. Working with APIs

- **Fetching Data**
 - Using `fetch` or `axios` to make API calls.
 - Handling loading and error states.
- **Displaying Data**
 - Rendering lists with `FlatList` and `SectionList`.
- **Real-Time Data**
 - Integrating WebSocket or Firebase for real-time updates.

7. Native Features

- **Platform-Specific Development**
 - Using `Platform` module for platform-specific code.
 - Styling and behavior differences between iOS and Android.
- **Device APIs**
 - Accessing device features like Camera, Location, Storage, and Notifications.
 - Using libraries like `react-native-permissions`, `react-native-camera`, and `react-native-geolocation`.

8. Animations

- **Basic Animations**
 - Using `Animated` API for simple animations.
 - Creating transitions and gestures.
- **Third-Party Libraries**
 - Using libraries like `react-native-reanimated` and `react-navigation-shared-element` for advanced animations.

9. Testing and Debugging

- **Debugging Tools**
 - Using React Native Debugger and Flipper.
 - Debugging with Chrome Developer Tools.
- **Testing**
 - Writing unit tests with Jest.
 - Testing components with React Native Testing Library.
 - End-to-end testing using Detox.

10. Performance Optimization

- **Improving Performance**
 - Optimizing lists with `FlatList` and `memo`.

- Avoiding unnecessary re-renders with `useMemo` and `useCallback`.
- **Code Splitting**
 - Lazy loading components and screens.
- **Profiling**
 - Using profiling tools for performance monitoring.

11. Deployment

- **Building and Publishing**
 - Generating APK/IPA files for Android/iOS.
 - Publishing apps to Google Play Store and Apple App Store.
- **Over-the-Air Updates**
 - Using Expo or CodePush for instant updates.

12. Advanced Topics

- **TypeScript with React Native**
 - Setting up TypeScript in a React Native project.
 - Using types and interfaces for props and state.
- **Native Modules**
 - Writing custom native modules for Android and iOS.
- **Third-Party Integrations**
 - Integrating libraries like Firebase, Stripe, and Mapbox.